# Guide: Containerized S3, MinIO, Parquet, and Delta Lake

1. Use a Self-Hosted S3-Compatible Storage

- MinIO: Lightweight, high-performance, S3 compatible. Runs in Docker.

- Ceph Object Gateway (RADOS): Enterprise-level storage.

- LocalStack: Testing S3 locally.

Example Docker command for MinIO:

```
docker run -p 9000:9000 -p 9001:9001 -d --name minio \
  -e "MINIO_ROOT_USER=admin" \
  -e "MINIO_ROOT_PASSWORD=password123" \
  -v /mnt/data:/data \
  minio/minio server /data --console-address ":9001"
```

2. Store Data in Optimized Formats

- Parquet: Columnar storage, good for analytics.

- Avro: Row-based, good for streaming.

- Delta Lake: Adds ACID transactions and versioning.

Python example to write Parquet to MinIO using pandas and s3fs:

```
import pandas as pd
import s3fs
df = pd.DataFrame({"id":[1,2,3],"name":["Alice","Bob","Charlie"],"score":[95,80,75]})
s3 = s3fs.S3FileSystem(key='admin',
secret='password123', client_kwargs={'endpoint_url':'http://localhost:9000'})
with s3.open('mybucket/data.parquet', 'wb') as f:
    df.to_parquet(f, engine='pyarrow', index=False)
```

3. Integrate with Supabase

- Store raw/analytical data in MinIO.

- Supabase stores metadata and lightweight references.

- Query large datasets with Python/Spark.

# Guide: Containerized S3, MinIO, Parquet, and Delta Lake

4. Delta Lake Option

```
from deltalake import DeltaTable, write_deltalake
import pandas as pd
df = pd.DataFrame({"id":[1,2], "val":[100,200]})
write_deltalake("s3://mybucket/delta_table", df, mode="overwrite",

storage_options={"key":"admin","secret":"password123","endpoint_url":"http://localhost:9000"})
```

5. Benefits

- Reduces database memory usage.
- Flexible formats allow compression and columnar storage.
- ACID-compliant Delta supports incremental updates.

Next Steps

1. Decide storage backend (MinIO is recommended).
2. Choose format: Parquet for analytics, Delta for ACID updates.
3. Use Supabase for transactional data.
4. Optionally use Spark or Python for processing large datasets.